

Vorlesung (WS 2016/17) *Softwarekonstruktion*

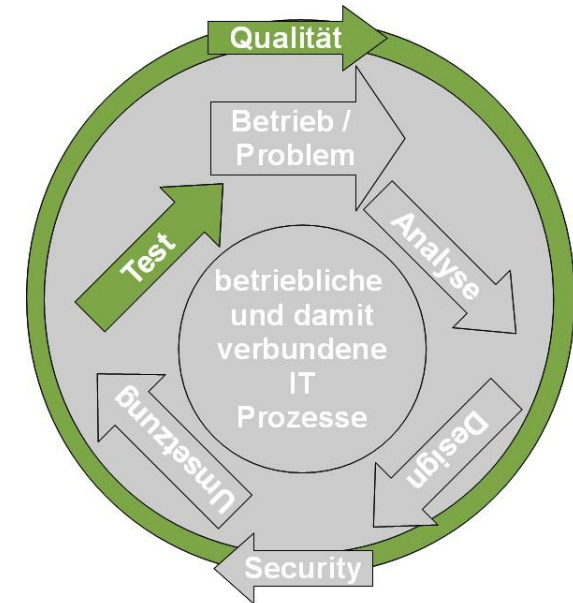
Dr. Boris Düdder

Lehrstuhl XIV – Software Engineering
TU Dortmund, Fakultät Informatik

Teil 2.1: Grundlagen der Softwareverifikation

v. 28.11.2015

- Modellgetriebene SW-Entwicklung
- Qualitätsmanagement
- **Testen**
 - **Grundlagen Softwareverifikation**
 - Softwaremetriken
 - Black-Box-Test
 - White-Box-Test
 - Testen im Softwarelebenszyklus



[Basierend auf dem Foliensatz „Basiswissen Softwaretest - Certified Tester“ des „German Testing Board“, 2011]

Literatur (s. Webseite):

- Andreas Spillner, Tilo Linz: Basiswissen Softwaretest.
- Eike Riedemann: Testmethoden für sequentielle und nebenläufige Software-Systeme.

- Bewusstsein für die Notwendigkeit von Verifikation von Software entwickeln.
- Verschiedene Konzepte und Methoden zur Softwareverifikation unterscheiden können.
- Ausgewählte Methoden der Verifikation anwenden und kombinieren können.

- **Vorheriger Abschnitt:** Softwarequalität
- **Dieser Abschnitt:** Testen der Software
 - Definition der Fehler
 - Positiv- & Negativ-Tests

2.1 Grundlagen der Software- verifikation

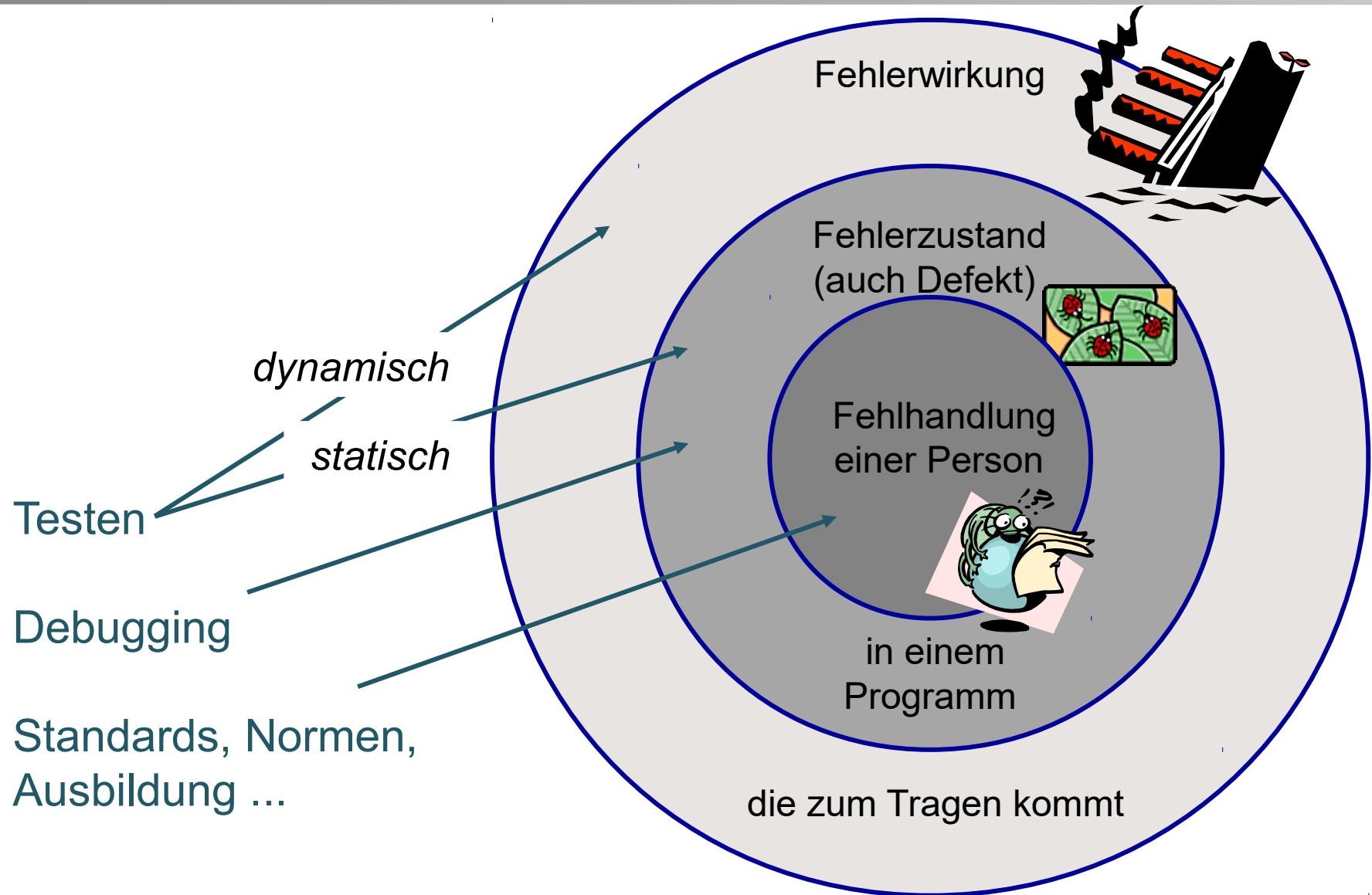


Fehler

Testen und Testfälle

Testprozess

Methoden der formalen Verifikation



Wiederholung: Verifizierung vs. Validierung ???

V?????ierung

Prüfung, ob Entwicklungsergebnis individuelle Anforderungen bezüglich einer speziellen beabsichtigten Nutzung erfüllt.

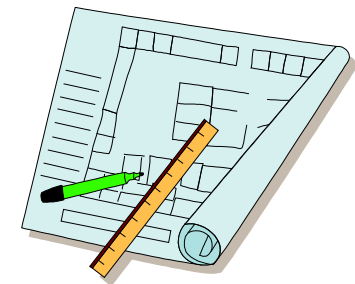
- Haben wir das **richtige System** realisiert?



V?????ierung

Prüfung, ob Ergebnisse einer Entwicklungsphase Vorgaben der Phaseneingangs-Dokumente erfüllen.

- Haben wir das System **richtig** realisiert?



Wiederholung: Verifizierung vs. Validierung

Validierung

Prüfung, ob Entwicklungsergebnis individuelle Anforderungen bezüglich einer speziellen beabsichtigten Nutzung erfüllt.

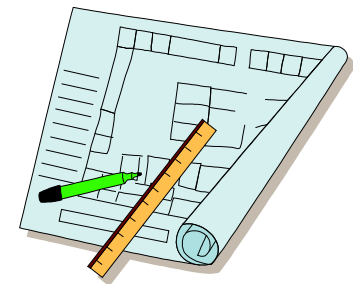
- Haben wir das **richtige System** realisiert?



Verifizierung

Prüfung, ob Ergebnisse einer Entwicklungsphase Vorgaben der Phaseneingangs-Dokumente erfüllen.

- Haben wir das System **richtig realisiert**?



2.1 Grundlagen der Software- verifikation



Fehler

Testen und Testfälle

Testprozess

Methoden der formalen Verifikation

Testen misst Qualität z.B. anhand Anzahl gefundener Fehlerwirkungen.

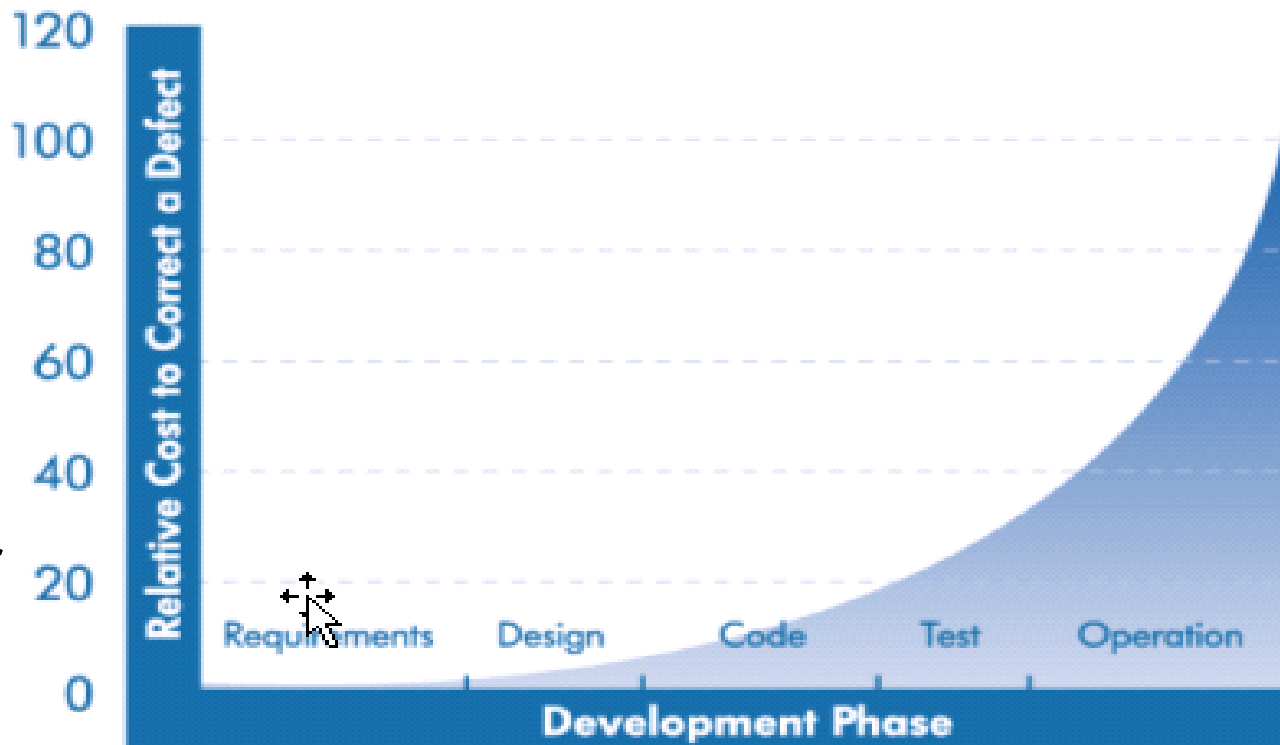
Erhöht indirekt

- **Qualität**, da Fehler(zustände) vor Auslieferung entdeckt und korrigiert werden können.
- **Prozessqualität**, da Fehler dokumentiert, analysiert und damit Fehlhandlungen in Zukunft vermieden werden können.
- **Vertrauen** in Qualität des Systems, wenn wenige oder keine Fehler gefunden werden.

- **Testaufwand** in Praxis:
 - 25% bis 50% des Entwicklungsaufwands
- **Testintensität** und **-umfang** in Abhängigkeit vom Risiko und Kritikalität festlegen.
- Fehler können **hohe Kosten** verursachen:
 - Geschätzte Verluste durch Softwarefehler in Mittelstands- und Großunternehmen in Deutschland ca. 84,4 Mrd. Euro p.a.
 - **Produktivitätsverluste** durch Computerausfälle aufgrund fehlerhafter Software ca. 2,6% des Umsatzes – 70 Mrd. Euro p.a.

Studie der LOT Consulting Karlsruhe, IT-Services 3/2001, S. 31

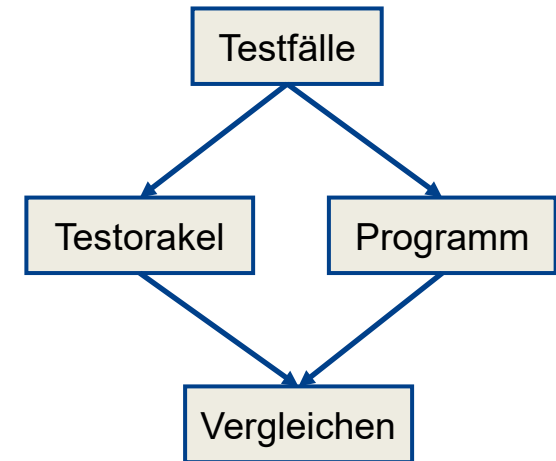
- **Frühzeitiges Prüfen:**
Fehler (zustände) früher erkennen und somit Kosten senken.



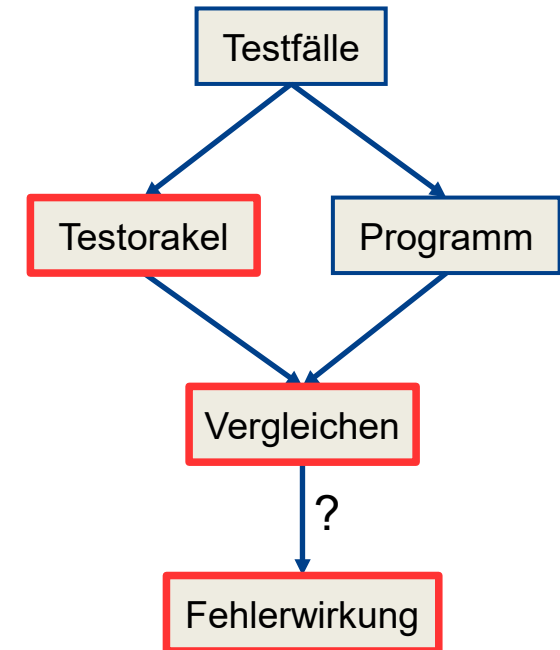
[„Finding and fixing a problem late in the development process can be 100 times more expensive than finding and fixing it during the requirement or design phase“. Barry Boehm: Software Engineering Economics (Prentice Hall, 1981)].

- **Erfolgreiches Testen** (Nachweis von Fehlerwirkungen) senkt (Gesamt-)Kosten.

- Ein **Testfall** besteht aus:
 - Eingabewert
 - Soll-Ergebnis
 - Vorbedingungen (vor Testfall erfüllt)
 - Nachbedingungen (nach Ablauf erfüllt)
- Wird der Testfall ausgeführt, zeigt das Testobjekt ein **Ist-Verhalten**.
 - Sind **Soll- und Ist-Verhalten** verschieden, liegt ggfs. **Fehlerwirkung** vor.
 - Prüfen, ob Fehler im Soll-Verhalten, Testfall, in der Spezifikation oder im Testobjekt liegt.
- **Testorakel** bestimmt Soll-Werte für jeden Testfall vor Testdurchführung.

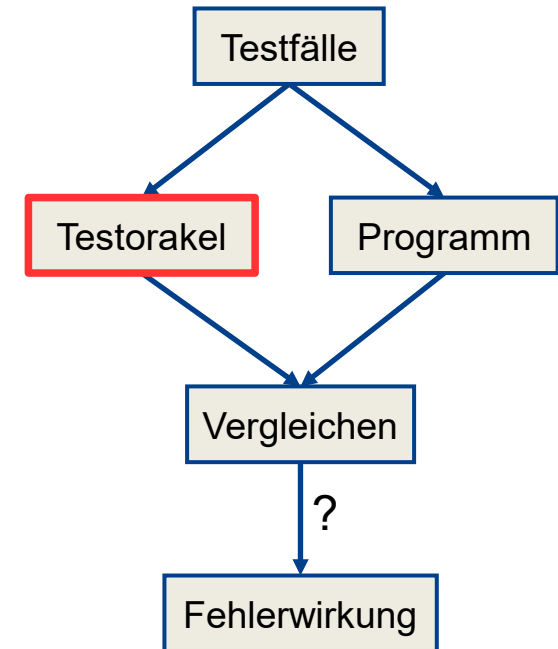


- Nach jedem durchgeführten Testfall prüfen, ob **Fehlerwirkung** vorliegt.
 - Vergleiche **beobachtetes** Ergebnis mit **erwartetem** Ergebnis.
 - **Soll-Verhalten** des Testobjekts für jeden Testfall vorab bestimmen.
 - Diese Information muss sich Tester bei Spezifikation eines Testfalls aus geeigneten Quellen beschaffen.



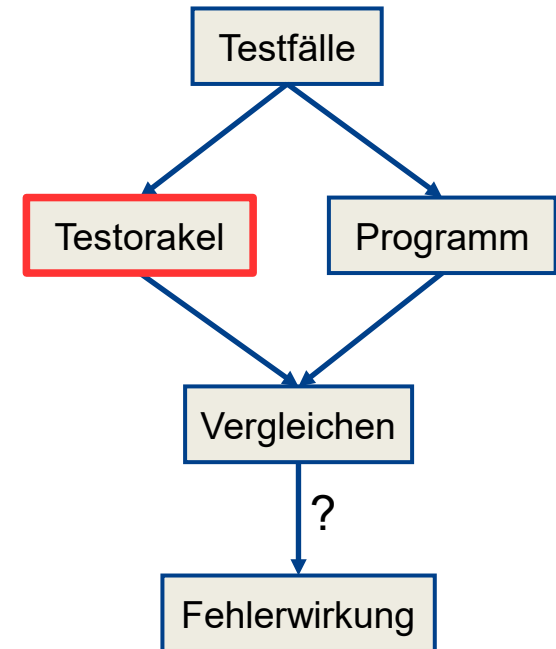
→ **Orakel / Testorakel:** Muss befragt werden und sagt jeweilige Soll-Ergebnisse vorher.

Welche Möglichkeiten können Sie sich für Testorakel vorstellen ?



Drei hauptsächliche Möglichkeiten:

- **Übliches Vorgehen** in Praxis: Soll-Datum aus Eingabedatum auf Grundlage der Spezifikation des Testobjekts ableiten.
- **Prototyp: z.B. werkzeuggestützte Erzeugung** durch formale Spezifikation. Prototyp Testorakel für Test des eigentlichen Programms.
- **Back-to-back-Test:**
 - Softwareprogramm wird **mehrfach** von unabhängigen Entwicklungsgruppen parallel erstellt.
 - **Programmversionen** mit Testdaten gegeneinander testen.
 - **Unterschiedliche** Ergebnisse → Fehlerzustand in einer Version.
 - Nur diejenigen Fehlerzustände unentdeckt, die in allen Versionen mit gleichen Fehlerwirkungen vorhanden.



2.1 Grundlagen der Software- verifikation



Fehler

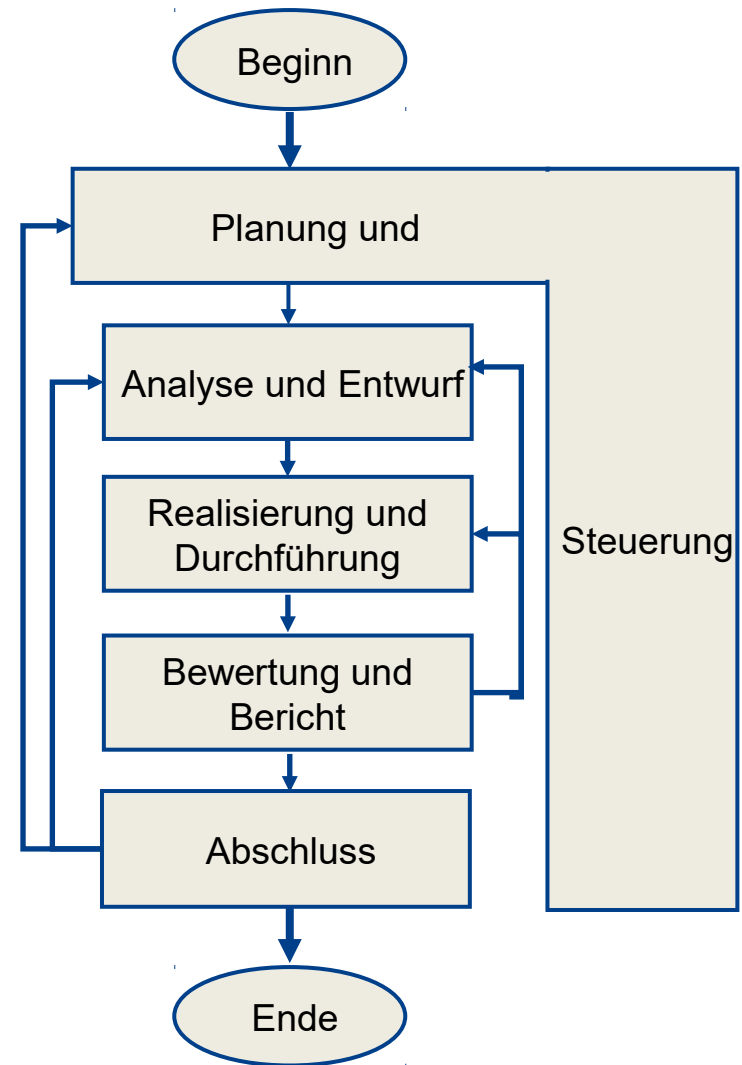
Testen und Testfälle

Testprozess

Methoden der formalen Verifikation

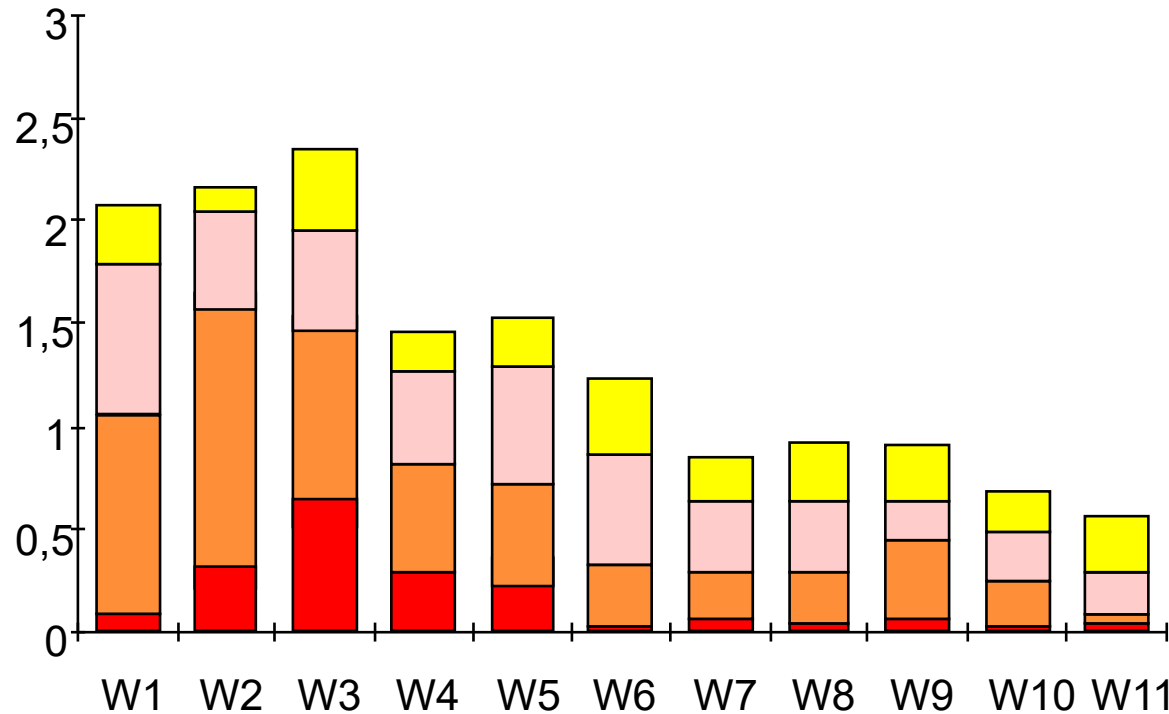
- **Testplanung** und Steuerung
- Testanalyse und **Testentwurf**
- **Testrealisierung** und Testdurchführung
- **Bewertung** von Ausgangskriterien (Test-Ende-Kriterien) und Bericht
- **Abschluss** der Testaktivitäten

- Werden z.T. zeitlich überlappend oder parallel ausgeführt.
- **Fundamentaler Testprozess:** Für jede Teststufe geeignet zu gestalten.



Beispiel für **Ausgangskriterien**: Gefundene Fehler nach Fehlerschweregrad

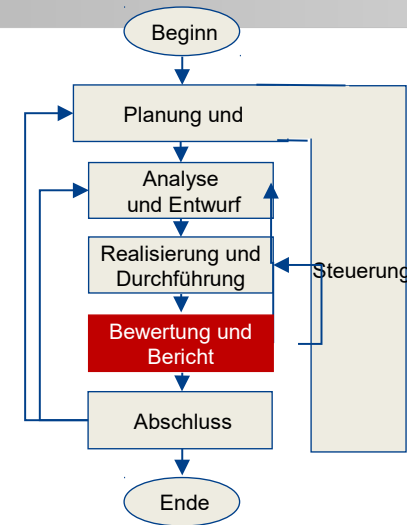
Neu / Teststunde

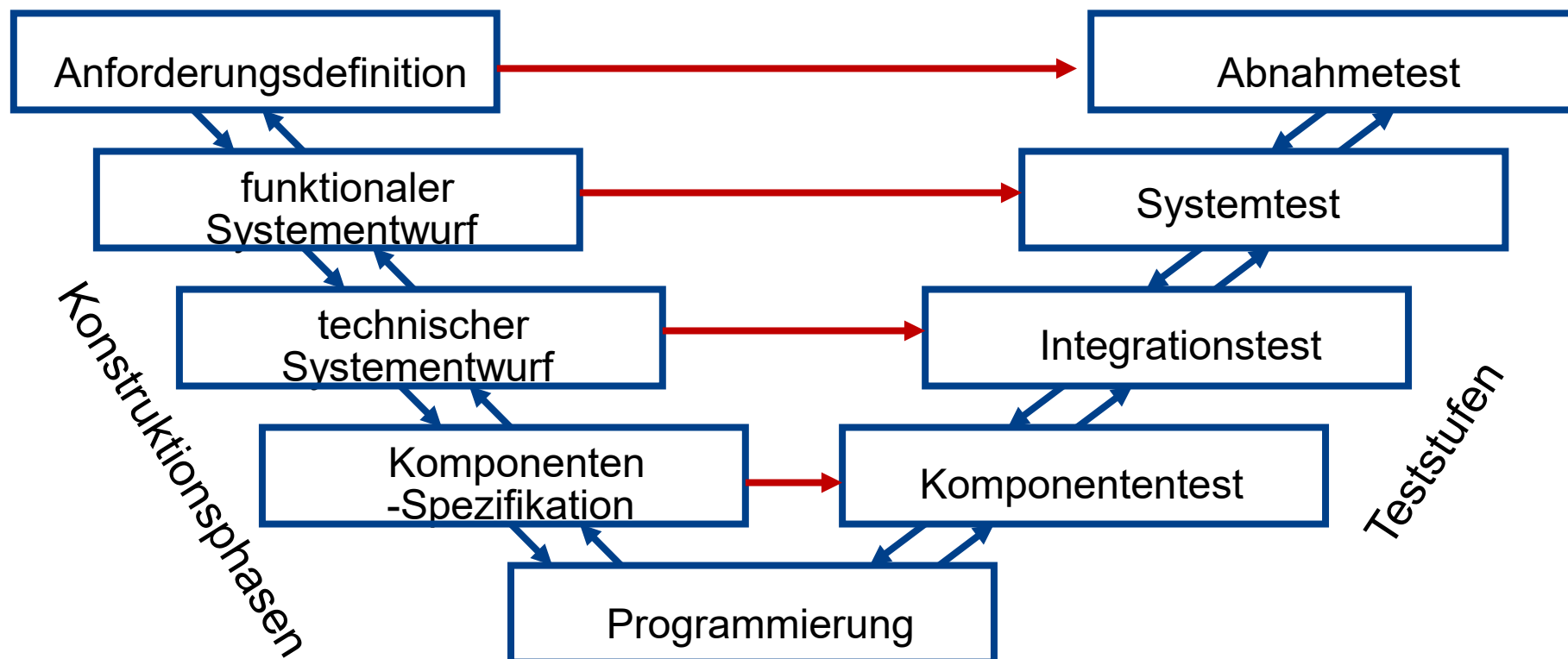


Fehlerschweregrad

- niedrig
- mittel
- hoch
- kritisch

Wx: Kalenderwoche





Legende
→ Testfälle basieren auf den entsprechenden Dokumenten

2.1 Grundlagen der Software- verifikation



Fehler

Testen und Testfälle

Testprozess

Methoden der formalen Verifikation

- Mathematische Beweise
 - Formale (strukturelle) operationelle Semantik (SOS) notwendig
 - Aber
 - Halteproblem
 - Gödelscher Unvollständigkeitssatz
 - Satz von Rice
 - Spezielle Kalküle wie Hoare-Kalkül
 - Theorembeweiser
- Model Checking

Nach Hoare, C.A.R., „An axiomatic basis for computer programming“. Communications of the ACM, Vol. 12, S. 567-583, 1969

Hoare Tripel $\{P\} S \{Q\}$

P , Q Zusicherungen: P Vorbedingung und Q Nachbedingung

Regelsystem der Form:

$$\frac{\begin{array}{c} \textit{Prämisse 1} \\ \textit{Prämisse 2} \\ \dots \\ \textit{Prämisse n} \end{array}}{\textit{Konklusion}}$$

Hoare Kalkül dient zum (konstruktiven) Beweisen von partieller und totaler Korrektheit eines Programms.

Leite rückwärts ausgehend von (gewünschten) Nachbedingungen $\rightarrow$ Vorbedingungen ab.

Vorgehen:

- 1) Nachbedingungen aufschreiben
- 2) Vorbedingungen notieren
- 3) Von unten nach oben Regeln anwenden (bzw. außen nach innen bei Schleifen)
- 4) Beweisskizze = Alle Anweisungen mit Hoare Klauseln versehen. Keine Kontradiktion $\rightarrow$ Beweis erbracht

Axiom der leeren Anweisung
$$\frac{}{\{P\} \textit{ skip } \{P\}}$$

skip ändert keine Variable

Zuweisungsaxiom
$$\frac{}{\{ P[E/x] \} x:=E \{P\}}$$

$P[E/x]$ = in P alle Vorkommen von x durch E substituiert

z.B.
$$\frac{}{\{ x+1 \leq N \} x:=x+1 \{x \leq N\}}$$

Kompositionsregel

$$\frac{\{P\} S \{R\}, \{R\} T \{Q\}}{\{P\} S; T \{Q\}}$$

Auswahlregel

$$\frac{\{P \wedge B\} S \{Q\}, \{P \wedge \neg B\} T \{Q\}}{\{P\} \text{ if } B \text{ then } S \text{ else } T \{Q\}}$$

In dieser Regel werden beide Zweige S und T bewiesen.

Iterationsregel

$$\frac{\{I \wedge B\} S \{I\}}{\{I\} \text{ while } B \text{ do } S \text{ done } \{I \wedge \neg B\}}$$

Konsequenzregel

$$\frac{P \Rightarrow P', \{P'\} S \{Q'\}, Q' \Rightarrow Q}{\{P\} S \{Q\}}$$

Vorbedingung gestärkt ($P \Rightarrow P'$)

Nachbedingung geschwächt ($Q' \Rightarrow Q$)

Partielle Korrektheit = Korrektheit ohne Beweis der tatsächlichen Terminierung

Totale Korrektheit = part. Korrektheit + sichere Terminierung

Nur partielle Korrektheit in Hoare-Kalkül beweisbar.
Warum?

Euklidischer Algorithmus zur ggT Berechnung

```
function ggT(x : integer; y : integer) : integer;  
// a, b : integer;
```

```
    a := x  
    b := y  
    while a != b do  
        while a > b do  
            a := a - b  
        while b > a do  
            b := b - a  
    return b;
```

1. Vor- und Nachbedingungen

$$\{x > 0 \wedge y > 0\}$$

a := x

b := y

while a != b do

 while a > b do

 a := a - b

 while b > a do

 b := b - a

$$\{a > 0 \wedge b = \mathbf{ggT}(x, y)\}$$

2. Obere Anweisungen

$\{x > 0 \wedge y > 0 \wedge \mathbf{ggT}(x, y) = \mathbf{ggT}(x, y)\}$

a := x

$\{a > 0 \wedge y > 0 \wedge \mathbf{ggT}(x, y) = \mathbf{ggT}(a, y)\}$

b := y

$\{a > 0 \wedge b > 0 \wedge \mathbf{ggT}(x, y) = \mathbf{ggT}(a, b)\}$

Zuweisungsaxiom

Invariante aus einer Überlegung: ein Teiler kann nicht 0 sein, da die Division durch 0 nicht definiert ist (liegen etwa zwei Primzahlen vor, ist der größte gemeinsame Teiler 1). Des Weiteren entspricht der gesuchte Teiler $\text{ggT}(x, y)$ immer $\text{ggT}(a, b)$. Hieraus wird die Invariante gebildet (B und $\neg B$).

Wie lautet Invarianten der äußeren Schleife?

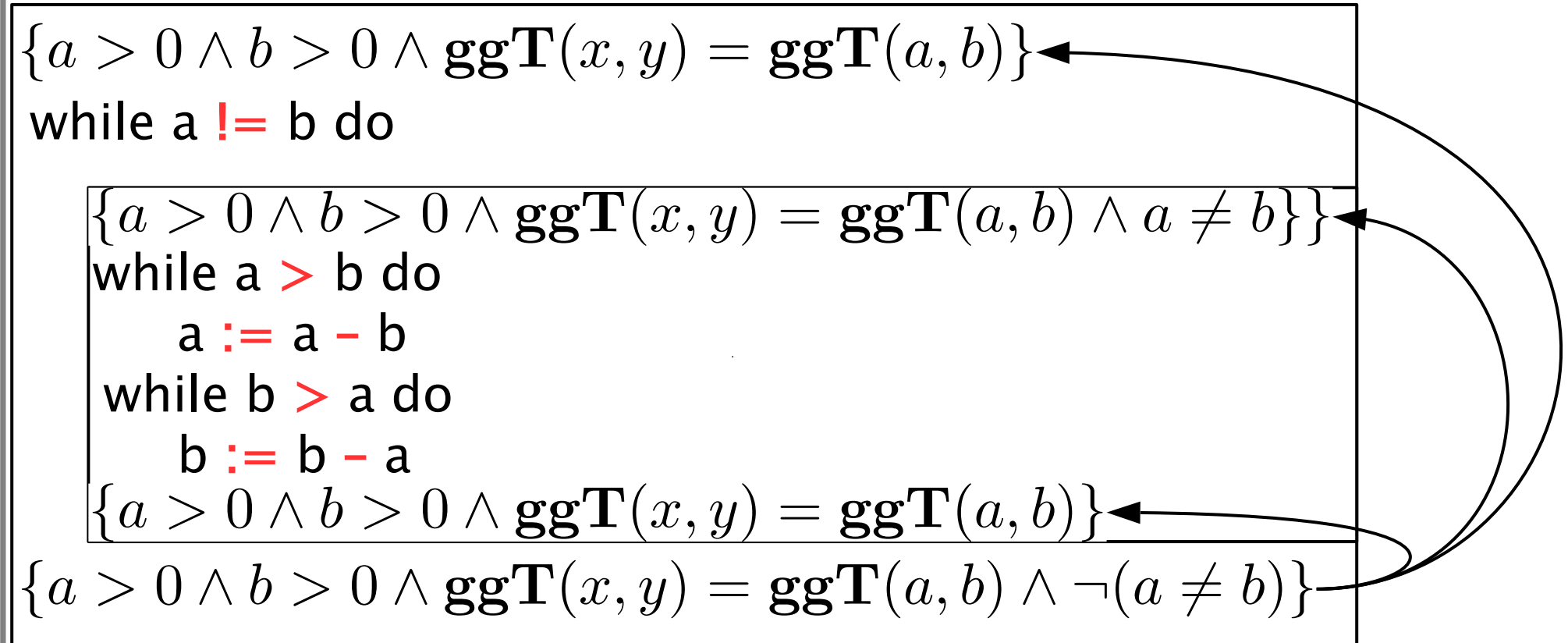
Invariante aus einer Überlegung: ein Teiler kann nicht 0 sein, da die Division durch 0 nicht definiert ist (liegen etwa zwei Primzahlen vor, ist der größte gemeinsame Teiler 1). Des Weiteren entspricht der gesuchte Teiler $\text{ggT}(x, y)$ immer $\text{ggT}(a, b)$. Hieraus wird die Invariante gebildet (B und $\neg B$).

$$I : a > 0 \wedge b > 0 \wedge \text{ggT}(x, y) = \text{ggT}(a, b)$$

$$I \wedge B : a > 0 \wedge b > 0 \wedge \text{ggT}(x, y) = \text{ggT}(a, b) \wedge a \neq b$$

$$I \wedge \neg B : a > 0 \wedge b > 0 \wedge \text{ggT}(x, y) = \text{ggT}(a, b) \wedge \neg(a \neq b)$$

3. Iterationsinvariante finden



Iterationsregel

- Von unten nach oben werden nun weiteren Anweisungen mit Hoare-Klauseln versehen.
- Überlegungen, aus der ersten While-Schleife, sind auch bei zweiten Schleife gültig.

3. Innere untere Schleife

$\{a > 0 \wedge b > 0 \wedge \mathbf{ggT}(x, y) = \mathbf{ggT}(a, b)\}$

while $b > a$ do

$\{a > 0 \wedge b > 0 \wedge \mathbf{ggT}(x, y) = \mathbf{ggT}(a, b) \wedge b > a\}$

$\{a > 0 \wedge b - a > 0 \wedge \mathbf{ggT}(x, y) = \mathbf{ggT}(a, b - a)\}$

$b := b - a$

$\{a > 0 \wedge b > 0 \wedge \mathbf{ggT}(x, y) = \mathbf{ggT}(a, b)\}$

$\{a > 0 \wedge b > 0 \wedge \mathbf{ggT}(x, y) = \mathbf{ggT}(a, b) \wedge \neg(b > a)\}$

Iterationsregel
Zuweisungsaxiom

3. Innere obere Schleife

$\{a > 0 \wedge b > 0 \wedge \mathbf{ggT}(x, y) = \mathbf{ggT}(a, b)\}$

while $b > a$ do

$\{a > 0 \wedge b > 0 \wedge \mathbf{ggT}(x, y) = \mathbf{ggT}(a, b) \wedge a > b\}$

$\{a - b > 0 \wedge b > 0 \wedge \mathbf{ggT}(x, y) = \mathbf{ggT}(a - b, b)\}$

$b := b - a$

$\{a > 0 \wedge b > 0 \wedge \mathbf{ggT}(x, y) = \mathbf{ggT}(a, b)\}$

$\{a > 0 \wedge b > 0 \wedge \mathbf{ggT}(x, y) = \mathbf{ggT}(a, b) \wedge \neg(a > b)\}$

Iterationsregel
Zuweisungsaxiom

Letzte innere While-Schleife (mit derselben Invariante)

- Jede Anweisung mit gültiger Formel versehen.
- Keine Kontradiktion $\rightarrow$ Beweis gültig

HAHA ist Lehrsystem für Hoare Logik (Uni Warschau)

- Download unter <http://www.mimuw.edu.pl/~tsznuk/haha/>
- Voraussetzungen:
 - Java 1.6
 - Microsoft Z3

Beispiel

```
function hello() : Z
  postcondition hello = 4
  hello := 2 + 2
```

- Vorteile der Methode:
 - Beweisbare Korrektheit
 - Einbettbar in Programmiersprachen (z.B. Assertions)
- Nachteil der Methode:
 - Triviale Programme → komplexe Beweise
 - Formelfindung nicht-automatisierbar (u.a. wg. Unentscheidbarkeit der Logik)
- Daher viele verschiedene Kalküle zur Programmverifikation, z.B. Reduktions-, Small- und Big-step Semantik

Büchi Automat $A = (Q, \Sigma, \Delta, I, F)$

- Q endliche Zustandsmenge
- Σ endliche Menge von Symbolen (Eingabealphabet)
- Δ Übergangsrelation mit $\Delta \subseteq Q \times \Sigma \times Q$
- I endliche Startzustandsmenge mit $I \subseteq Q$
- F ist eine endliche Endzustandsmenge mit $F \subseteq Q$

A **akzeptiert** gdw. ein Endzustand wird unendlich oft durchlaufen.

Sprache eines Büchi Automaten A :

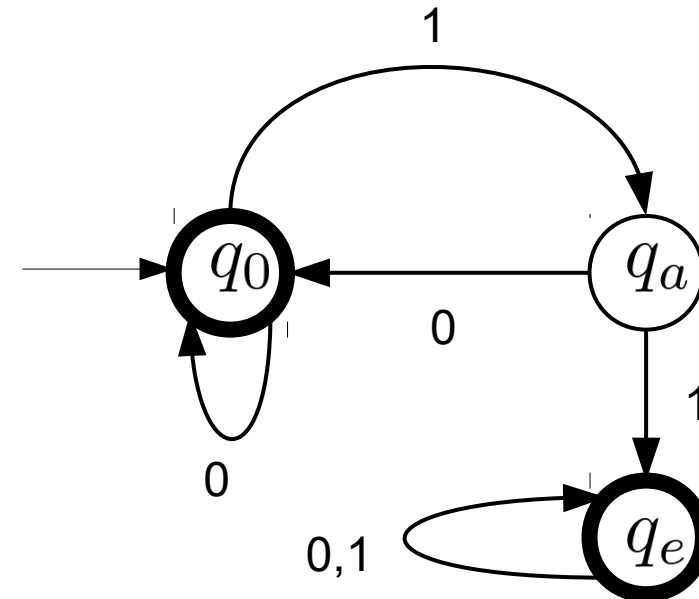
$$Lang(A) = \{w \in \Sigma^\omega \mid w \text{ wird von } A \text{ akzeptiert}\}$$

Automat der unendliche Eingabe liest und darin überprüft ob auf jede 1 direkt immer eine 0 folgt.

$$A = (\{q_0, q_a, q_e\}, \{0, 1\}, \Delta, \{q_0\}, \{q_0, q_e\})$$
$$\Delta = (q_0, 0, q_0) \cup (q_0, 1, q_a) \cup (q_a, 0, q_0) \cup (q_a, 1, q_e) \cup \\ (q_e, 0, q_e) \cup (q_e, 1, q_e)$$

Startzustand q_0

Fehlerzustand q_e



Monadische Logik zweiter Stufe (MSO)

- Prädikatenlogik zweiter Stufe mit Einschränkung: Quantifizierung über Relationen auf einstellige Relationen beschränkt.
- Generalisiert temporale Logiken

Model-checking in MSO entscheidbar aber von hoher Komplexität

Satz von Büchi-Elgot-Trakhtenbrot:

Endliche Automaten und *MSO Formeln* sind äquivalent.

Idee: Spezifikation von System M und zu prüfende Eigenschaft $\varphi \rightarrow$ Büchi-Automaten $B(M)$ und $B(\varphi)$

M erfüllt φ gdw. $B(M)$ ist Teilmenge der zulässigen durch $B(\varphi)$ beschriebenen Systementwicklungen.

$$Lang(B(M)) \subseteq Lang(B(\varphi))$$



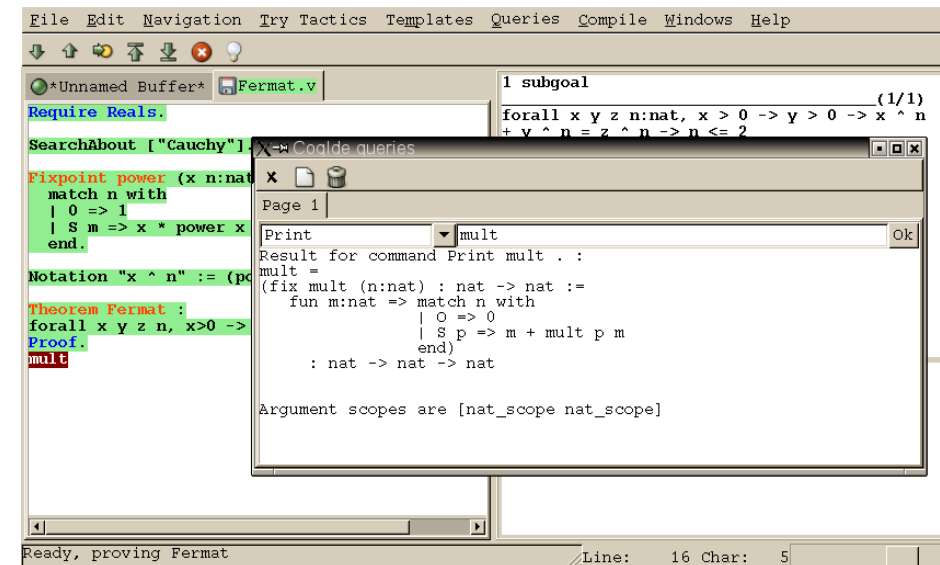
$$Lang(B(M)) \cap \overline{Lang(B(\varphi))} = \emptyset$$



$$Lang(B(M)) \cap Lang(B(\neg\varphi)) = Lang(B(M) \cap B(\neg\varphi)) = \emptyset$$

Theorembeweiser beweist Aussagen über formale Strukturen durch Anwendung von Regeln.

- Typen und (algebraische) Datentypen (natürliche Zahlen, Listen, Paare, . . .)
- Mengen, Relationen, Funktionen
- funktionale Programmierung
- definiert im jeweiligen System!
- Coq, Isabelle, Agda, HOL, NuPrl, etc.



```
File Edit Navigation Try Tactics Templates Queries Compile Windows Help
*Unnamed Buffer* Fermat.v
Require Reals.
SearchAbout [\"Cauchy\"]
Fixpoint power (x : nat) (n : nat) : nat :=
  match n with
  | 0 => 1
  | S m => x * power x m
end.
Notation \"x ^ n\" := (power x n)
Theorem Fermat :
  forall x y z n, x > 0 ->
  Proof.
mult

1 subgoal
forall x y z n : nat, x > 0 -> y > 0 -> x ^ n + y ^ n = z ^ n -> n <= 2

Print mult
Result for command Print mult . :
mult =
(fun m : nat => match n with
  | 0 => 0
  | S p => m + mult p m
end)
: nat -> nat -> nat
Argument scopes are [nat_scope nat_scope]
```

Quelle: coq.inria.fr

In diesem Abschnitt:

- Fehlerwirkung - **Nichterfüllung** einer Anforderung!
 - Beschreibung dieses Sachverhalts als Fehlerwirkung (*failure*).
 - Ursache: Fehlerzustand (*fault*) in Software.
 - die durch Fehlhandlung (*error*) einer Person verursacht wurde.
- Software-Prüfung: **Wichtige Maßnahmen** zur Sicherung der Softwarequalität
- Ansätze der formalen Methoden der Softwareverifikation

Im nächsten Abschnitt:

- Softwariemetriken und Schätzung

Anhang (zusätzliche Einzelheiten und Beispiele)

Softwarekonstruktion
WS 2016/17



- Ein großer Teil der Folien wurde übernommen aus einer gemeinsamen Hochschul-übergreifenden Lehrveranstaltung und vom GTB entsprechend des derzeitigen Syllabus 2011 aktualisiert.
- Dank gilt den Kollegen:
 - Dr. Falk Fraikin, TU Darmstadt
 - Dr. Eike Hagen Riedemann, ehemals TU Dortmund
 - Prof. Dr. Andreas Spillner, Hochschule Bremen
 - Prof. Dr. Mario Winter, FH Köln
- HS@GTB bezeichnet die Hochschul-Mitglieder des German Testing Boards (Stand 07.2011):
 - Dipl.-Inf. Timea Illes-Seifert, EnBW SIS GmbH
 - Dipl.-Inf. Horst Pohlmann, Hochschule Ostwestfalen-Lippe
 - Prof. Dr.-Ing. Ina Schieferdecker, FU Berlin
 - Prof. Dr. Mario Winter, FH Köln

aktuelle Informationen unter <http://www.german-testing-board.info>

Fehler:

- **Nichterfüllung** festgelegter Anforderung.
- **Abweichung** zwischen Ist-Verhalten und Soll-Verhalten.

Mangel:

- Gestellte Anforderung oder berechnigte **Erwartung** in Bezug auf beabsichtigten Gebrauch **nicht angemessen erfüllt**.
- Z.B. Beeinträchtigung der Verwendbarkeit bei gleichzeitiger Erfüllung der Funktionalität oder Nichterfüllung angemessener Erwartung.

- Jeder Fehler oder Mangel: seit Zeitpunkt der Fertigstellung in Software vorhanden.
→ Kommt erst bei Ausführung der Software zum Tragen.
- Beschreibung dieses Sachverhalts als **Fehlerwirkung** (*failure*).
- Ursache einer Fehlerwirkung: **Fehlerzustand** (*fault*) in Software (auch als Defekt, innerer Fehler bezeichnet).
- Ursache eines Fehlerzustands: **Fehlhandlung** (*error*) einer Person.
- Beachte **Fehlermaskierung**: »Ein Umstand, bei dem ein Fehlerzustand die Aufdeckung eines anderen verhindert« [IEEE 610].

Angelehnt an: DIN 66271:1995

Informationstechnik - Software-Fehler und ihre Beurteilung durch Lieferanten und Kunden, Beuth Verlag, Berlin, 1995

- **Fehlhandlung (error)**

1. **Menschliche Handlung** (des Entwicklers), die zu einem Fehlerzustand in der Software führt.
2. Menschliche Handlung (des Anwenders), die **unerwünschtes Ergebnis** (im Sinne von Fehlerwirkung) zur Folge hat (Fehlbedienung).
3. Unwissentlich, versehentlich oder absichtlich ausgeführte **Handlung oder Unterlassung**, die unter gegebenen Umständen (Aufgabenstellung, Umfeld) dazu führt, dass geforderte Funktion eines Produkts beeinträchtigt ist.



- **Fehlerzustand / Defekt (fault)**

- **Inkorrektes Teilprogramm** (z.B. mit inkorrektter Anweisung oder Datendefinition), das Ursache für Fehlerwirkung sein kann.
- Zustand eines (Software-)Produkts oder einer seiner Komponenten, der unter spezifischen Bedingungen geforderte Funktion des Produkts beeinträchtigen kann bzw. zur Fehlerwirkung führt.



- **Fehlerwirkung (failure)**

- Wirkung eines Fehlerzustands, die bei Ausführung eines Programms nach »außen« in Erscheinung tritt.
- **Abweichung** zwischen (spezifizierten) Soll-Wert und (beobachtetem) Ist-Wert (bzw. Soll- und Ist-Verhalten).
- Abweichung einer Komponente/eines Systems von erwarteter Lieferung, Leistung oder dem Ergebnis [nach Fenton].



•Anmerkung: Fehlerwirkungen können (selten) durch Höhenstrahlung, elektromagnetische Felder oder Hardwarefehler hervorgerufen werden.

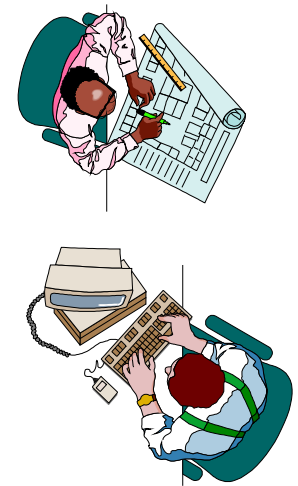
•Anmerkung: So kann Fehlerwirkung zur langsamen Ausführung, zu inkorrekten Ausgaben oder zum Abbruch der Ausführung führen.

Prozess, der sich (sowohl statisch als auch dynamisch) mit

- **Planung**
- **Vorbereitung**
- **Bewertung** einer Software
- hierzu in Beziehung stehenden **Arbeitsergebnissen** befasst.

Um Software mit dem **Ziel** zu bewerten, dass diese

- allen festgelegten Anforderungen
- ihren Zweck erfüllt und
- etwaige Fehlerzustände zu finden.



Grundsätze zum Testen in letzten 40 Jahren und können als Leitlinien dienen:

- Grundsatz 1: »**Testen zeigt die Anwesenheit von Fehlern**«
 - Nachweisen der Anwesenheit von Fehlerwirkungen.
 - Kann nicht zeigen, dass keine Fehlerzustände im Testobjekt vorhanden sind !
 - »Program testing can be used to show the presence of bugs, but never to show their absence!« Edsger W. Dijkstra, 1970
- Grundsatz 2: »**Vollständiges Testen ist nicht möglich**«
 - Vollständiges Testen – Austesten – nicht möglich.

- Grundsatz 3: »**Mit dem Testen frühzeitig beginnen**«

- **Frühzeitiges Prüfen:** Fehler(zustände) früher erkennen und somit Kosten senken.

[„Finding and fixing a problem late in the development process can be 100 times more expensive than finding and fixing it during the requirement or design phase“. Barry Boehm: Software Engineering Economics (Prentice Hall, 1981)].

- Grundsatz 4: »**Häufung von Fehlern ("Fehlercluster")**«

- Testaufwand: Proportional zur erwarteten und später beobachteten Fehlerdichte auf Module fokussieren.
- Kleiner Teil der Module enthält meiste Fehlerzustände.
 - Während Testphase entdeckt.
 - Für meisten Fehlerwirkungen im Betrieb verantwortlich.

- Grundsatz 5: »**Wiederholungen haben keine Wirksamkeit**«
 - Tests nur zu wiederholen, bringt keine neuen Erkenntnisse.
 - Testfälle sind zu prüfen, zu aktualisieren und zu modifizieren.
- Grundsatz 6: »**Testen ist abhängig vom Umfeld**«
 - Vom Umfeld abhängig.
 - Sicherheitskritische Systeme intensiver testen
- Grundsatz 7: »**Trugschluss: Keine Fehler heißt, dass das System brauchbar ist**«
 - System ohne Fehlerwirkungen $\neq$ System entspricht den Vorstellungen der späteren Nutzer.

- »Testen ist ökonomisch sinnvoll, solange **Kosten** für Finden und Beseitigen eines Fehlers im Test niedriger sind als Kosten, die mit Auftreten eines Fehlers bei **Nutzung** verbunden sind.«

M. Pol, T. Koomen, A. Spillner: Management und Optimierung des Testprozesses. dpunkt-Verlag, 2. Auflage, 2002

- »Guter Test ist wie Haftpflichtversicherung: Kostet richtig Geld, lässt aber Projektleiter und Kunden ruhig schlafen. Zum guten Schlaf gehört auch gute Versicherung, die alle möglichen Risiken abdeckt. Zum Vertrauen in Software gehört guter Test, der ganze Produktionswirklichkeit abdeckt.«

Siedersleben, J. (Hrsg): Softwaretechnik, Praxiswissen für Softwareingenieure. 2. Auflage, Hanser, 2003

- **Erfolgreiches Testen** (Nachweis von Fehlerwirkungen) senkt (Gesamt-)Kosten.

- **Testaufwand** in Praxis:
 - 25% bis 50% des Entwicklungsaufwands
- **Testintensität** und **-umfang** in Abhängigkeit vom Risiko und Kritikalität festlegen.
- Fehler können **hohe Kosten** verursachen:
 - Geschätzte Verluste durch Softwarefehler in Mittelstands- und Großunternehmen in Deutschland ca. 84,4 Mrd. Euro p.a.
 - **Produktivitätsverluste** durch Computerausfälle aufgrund fehlerhafter Software ca. 2,6% des Umsatzes – 70 Mrd. Euro p.a.

Studie der LOT Consulting Karlsruhe, IT-Services 3/2001, S. 31

- Testen unterliegt immer **beschränkten Ressourcen** (besonders Zeit, wenn Testen erst am Ende der Entwicklung in Angriff genommen wird)
- Besonders wichtig:
 - **Adäquate** (zum Testobjekt und den Qualitätszielen passende) Testverfahren auswählen.
 - **Unnötige Tests** (die keine neuen Erkenntnisse liefern) vermeiden.
 - Sowohl **Positiv-Tests** als auch **Negativ-Tests** berücksichtigen.
 - Auch Tests auf **Funktionalität**, die nicht gefordert ist, können sinnvoll sein.

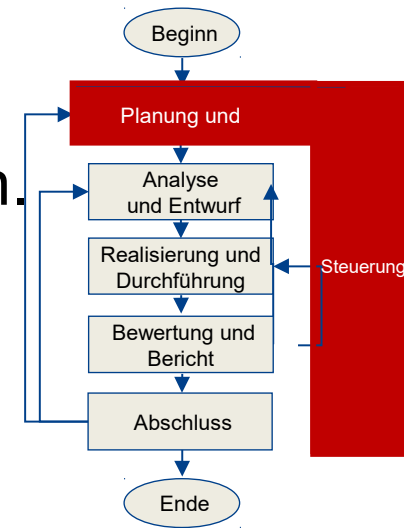
- **Benutzungshandbuch** verwenden
- System selbst (Nutzungsprofil erstellen)
→ **schlechteste Möglichkeit**
- (Getestete!) Vorgängerversionen, Programme mit ähnlicher Funktionalität
- Nicht immer lassen sich exakte Werte vorhersagen oder ermitteln. Toleranzbereich festlegen, Plausibilität prüfen
- **Erfahrung ist wichtig!**

- **Limitierte Ressourcen** (Zeit und Personal) erfordern, dass wichtigste Testfälle zuerst durchgeführt werden!
- Kriterien für Priorisierung:
 - Erwartete Fehlerschwere bzw. Höhe des Schadens
 - **Eintrittswahrscheinlichkeit** einer Fehlerwirkung
 - Kombinierte Betrachtung von Schwere und Eintrittswahrscheinlichkeit (Risiko = Schadenshöhe * Eintrittswahrscheinlichkeit)
 - **Wahrnehmung** der Fehlerwirkung
 - Priorität der Anforderungen durch Kunden
 - Gewichtung der Qualitätsmerkmale durch Kunden
 - **Priorität der Testfälle** aus Sicht der Entwicklung (schwerwiegende Auswirkungen oder/und Komplexes zuerst)
 - **Hohes Projektrisiko**
 - Wo viele Fehler sind, finden sich meist noch mehr.
→ Möglichkeit zur Änderung der Priorität.

- Eng verzahnt mit Softwareentwicklung.
- **Eigenständiger Prozess.**
- **Verfeinerter Ablaufplan** für Tests jeder Teststufe notwendig.
- Entwicklungsaufgabe »Test« ist in kleine Arbeitsabschnitte (Aktivitäten) aufzuteilen.

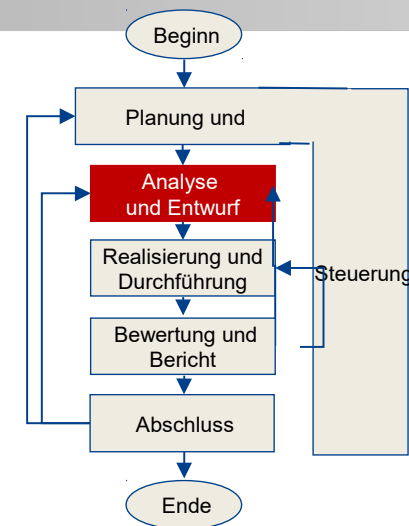
Testaufgaben: Testplanung und Steuerung (2)

- Grobe **Zeitplanung** festlegen
 - Testanalyse und -spezifikationsaufgaben terminieren.
 - Testrealisierung, -durchführung und -auswertung terminieren.
- **Teststeuerung:**
 - Messen und analysieren der Resultate.
 - Überwachen und Dokumentieren von Testfortschritt, erreichter Testabdeckung und der Ausgangskriterien (Testendekriterien).
 - Anstoß von Korrekturmaßnahmen.
 - Anpassung der Zeit- und Ressourcenmaßnahmen.
 - Treffen von Entscheidungen.



Testaufgaben: Testanalyse und -entwurf (1)

- Allgemeine Testziele werden zu konkreten Testbedingungen und Kriterien detailliert.
- Grundlage für die Überlegungen sind alle Dokumente, aus denen Anforderungen an das Testobjekt hervorgehen (Testbasis).
- **Review** der Testbasis (Anforderungen, Architektur, Entwurf, Schnittstellen, ...).
- **Testbarkeit** von Anforderungen und System bewerten.
- Identifizierung von **Testbedingungen** / **Testanforderungen**.
- Entwurf der Testumgebung (Infrastruktur, Werkzeuge, ...).
- Unter Anwendung der gewählten Testentwurfsverfahren sind die entsprechenden Testfälle zu spezifizieren.
- Unterscheidung zwischen abstrakten (logischen) und konkreten Testfällen.
- Sicherstellung der **Rückverfolgbarkeit** zwischen Testbasis und Testfällen in beiden Richtungen.

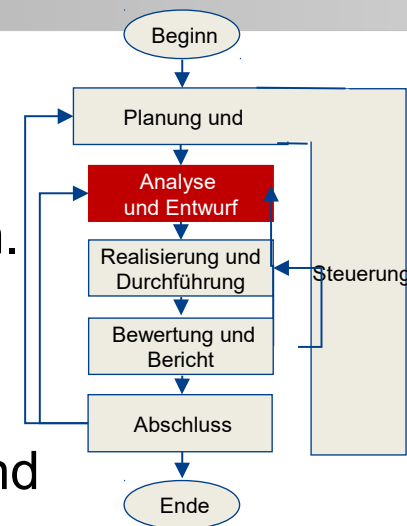


Testaufgaben: Testanalyse und -entwurf (2)

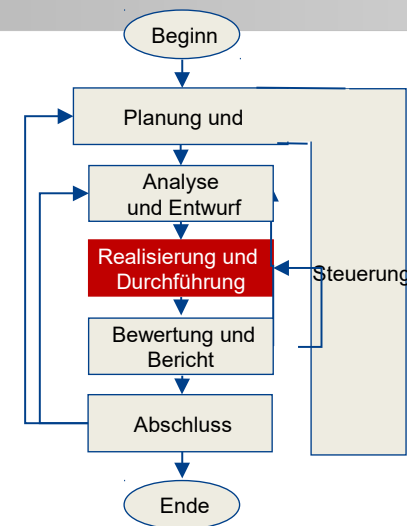
- Testfälle umfassen mehr als nur Testdaten (die Eingaben)!
 - **Ausgangsbedingung** (Vorbedingung), die vor der Durchführung erfüllt sein muss, z.B. Zustand des Testobjekts und globaler Daten.
 - Vor der Testdurchführung ist festzulegen, welches Ergebnis bzw. welches Verhalten erwartet wird (Soll- Ergebnis bzw. -Verhalten).
 - **Nachbedingung**, die nach dem Test erfüllt sein muss, z.B. Zustand des Testobjekts und der Datenbank.

Anmerkung: Zudem gelten **Randbedingungen**, die einzuhalten sind, z.B. Netzwerkstatus.

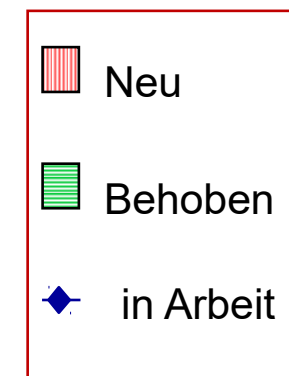
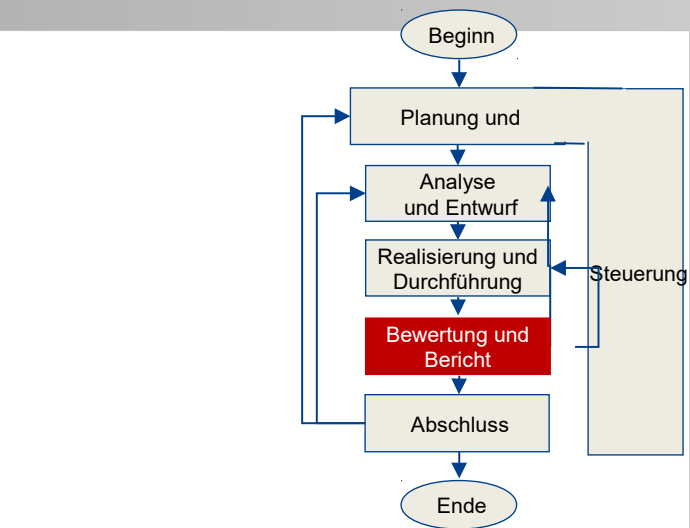
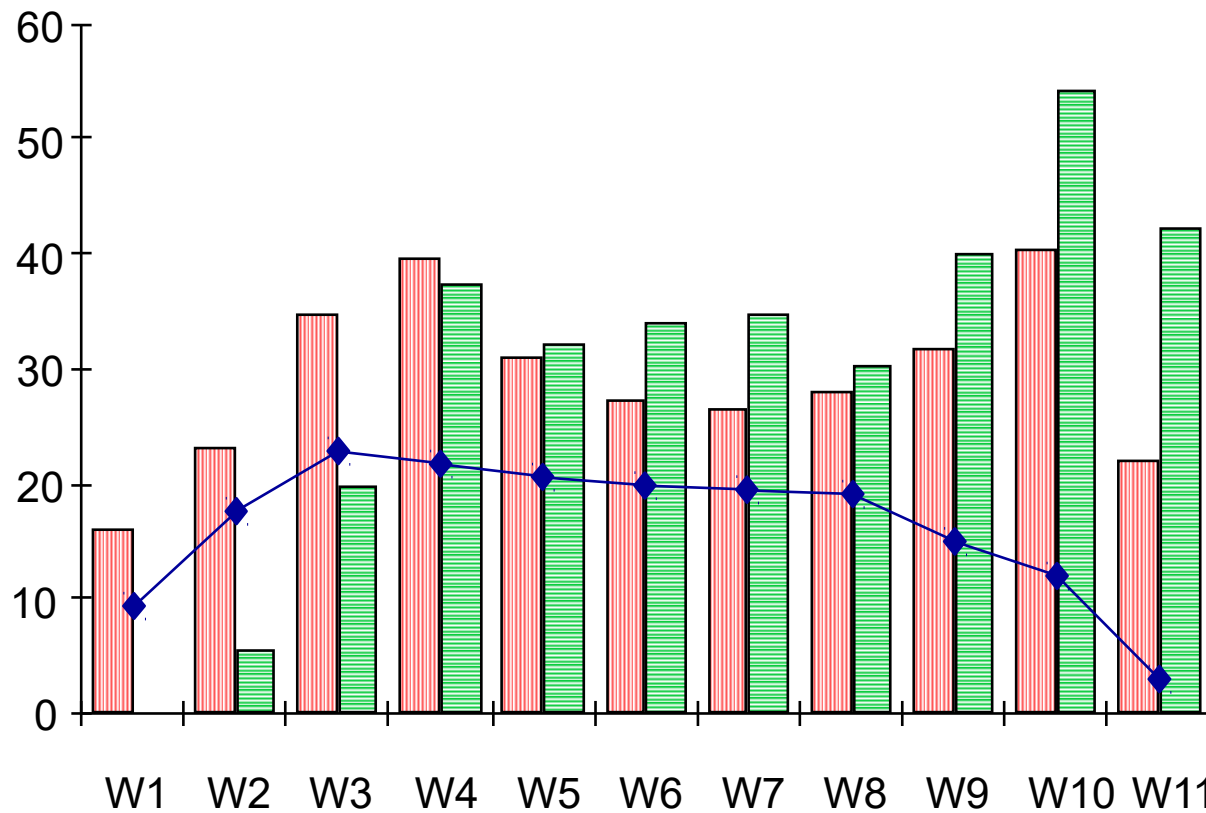
- Testfälle werden in der Testrealisierung teilweise zu **Testsuiten** (Testmengen, Testablaufspezifikationen) verknüpft:
 - Jeder Testfall sollte nur einen oder wenige elementare Bearbeitungsschritte oder **Systemaufrufe** (Testschritte) umfassen.
 - Schon beim Testentwurf darauf achten, welche Nachbedingungen von Testfällen welche Vorbedingungen von Testfällen erfüllen.



- Konkretisierung und **Priorisierung der Testfälle**.
- Erstellung von Testdaten und Testszenarien.
- Zusammenstellung der Testsuiten.
- **Vorbereitung der Testrahmen** und Entwicklung von Skripten zur Testautomatisierung.
- Kontrolle, ob das Testsystem korrekt aufgesetzt wurde und **Sicherstellung der richtigen Konfigurationen**.
- Vorab: Prüfung auf **Vollständigkeit** der zu testenden Teile.



Beispiel für **Ausgangskriterien:** Vergleich gefundene / behobene Fehler

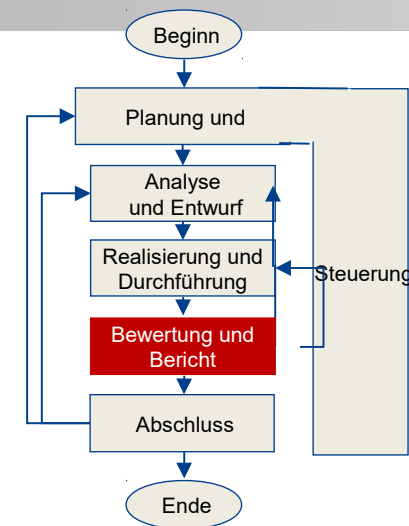


Wx: Kalenderwoche

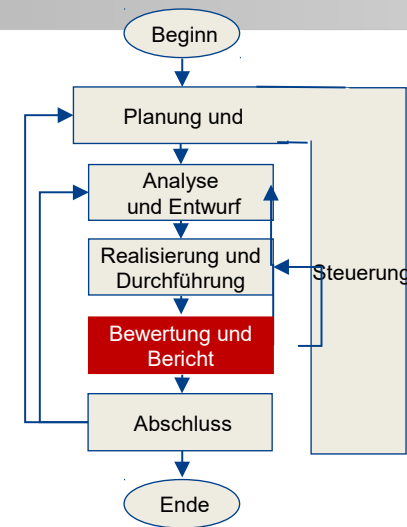
- Ergebnisse der Testdurchführung mit definierten Zielen des Tests vergleichen.
- Fehler gefunden ? Fehlerwirkung nachweisbar ?
Nichtübereinstimmung von Soll- und Ist-Ergebnis !
- Aber: Es kann auch
 - das Soll-Ergebnis
 - die Testumgebung
 - die Testfallspezifikation

falsch bzw. fehlerhaft sein.

- **Fehlerschweregrad** (Fehlerklasse) bestimmen, Priorität für die Beseitigung festlegen
 - Testende erreicht ?
 - Überdeckungsgrad erreicht ?
 - Problem: unerreichbarer Programmcode

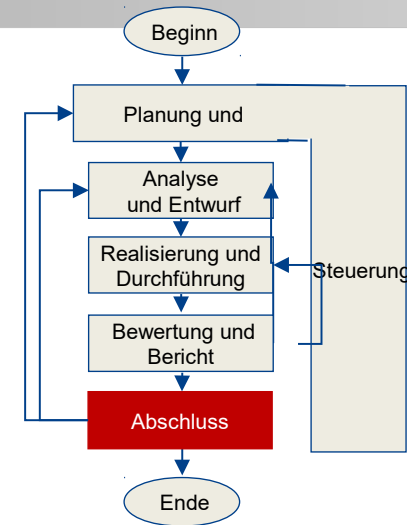


- Aus dem **Testprotokoll** muss hervorgehen, welche Teile wann, von wem, wie intensiv und mit welchem Ergebnis getestet wurden.
- Anhand dieser Testprotokolle muss die **Testdurchführung** für nicht direkt beteiligte Personen (Kunden) nachvollziehbar sein.
- Es muss nachweisbar sein, ob die geplante **Teststrategie** tatsächlich umgesetzt wurde.
- **Auswertung der Testprotokolle** in Hinblick auf die im Testplan festgelegten Ausgangskriterien (Testendekriterien).
- Weiterer Aufwand gerechtfertigt ? Faktoren für das Testende in der Praxis: **Zeit und Kosten**.
- Entscheidung, ob mehr Tests durchgeführt oder ob die festgelegten **Ausgangskriterien** angepasst werden müssen.



Testaufgaben: Abschluss der Testaktivitäten (1)

- Daten von abgeschlossenen Testphasen sammeln und konsolidieren (Erfahrungen, Testmittel, Fakten, Zahlen, ...).
- **Kontrolle**, welche geplanten Arbeitsergebnisse geliefert wurden.
- Schließung der Fehler/Abweichungsmeldungen oder Erstellung von **Änderungsanforderungen** für weiter bestehende Fehler/ Abweichungen.
- **Dokumentation** der Abnahme des Systems.



Testaufgaben: Abschluss der Testaktivitäten (2)

- **Dokumentation und Archivierung** der Testmittel, Testumgebung und der Infrastruktur.
- **Konservierung** der Testware und Übergabe an die Wartungsorganisation
 - Während der Wartung und Pflege soll wieder alles benutzbar sein.
 - Muss daher übertragbar und aktualisierbar sein.
- **Analyse und Dokumentation** von »*lessons learned*« für spätere Projekte und Verbesserung der Testreife:
 - Evaluation des Testprozesses
 - Einschätzung des Testprozesses
 - Erkennen von Verbesserungspotential
- **Verbesserung des Testprozesses** durch Umsetzung der Erkenntnisse.

